BCA40107

Programme Name and Semester:-
Bachelor of Computer Applications (BCA) - Semester IV

Course Name (Course Code):-
Object Oriented Programming in JAVA (BCA40107)

Academic Session:- **2025-2026**

Study Material

(Object Oriented Programming in JAVA - BCA40107)

Table of Contents

Module 1: Object Oriented Design

1. Concepts of Object-Oriented Programming Language (OOP) 3
- 1.1 What is OOP? 3
- 1.2 Key Ideas Behind OOP 4
- 1.3 Characteristics of an Object-Oriented Language 4

2. Object 5
- 2.1 Definition 5
- 2.2 State, Behavior, Identity 5
- 2.3 Examples 5

3. Class 6
- 3.1 Definition 6
- 3.2 Analogy 6
- 3.3 Code-Level Example 6
- 3.4 Types of Classes (Brief Idea) 8

4. Relationships Among Objects 8
- 4.1 Types of Object-Level Relationships 8
- 4.2 Association Between Objects 10
- 4.3 Aggregation / Whole-Part Between Objects 10
- 4.4 Dependency Between Objects 10
- 4.5 Interaction Through Message Passing 11

5. Relationships Among Classes (Class-Level Relationships) 11
- 5.1 Association 12
- 5.2 Aggregation 12
- 5.3 Aggregation Using Instantiation 13
- 5.4 Composition (Strong Aggregation) – for Contrast 13
- 5.5 Instantiation Relationship 13
- 5.6 Meta-Class 14

Name of the Faculty:- **Tridib Paul**
Designation and Department:- **Assistant Professor (Computational Sciences)**
Brainware University, Kolkata

5.7 Other Class Relationships (Brief) 15
6. Differences Between OOP and Procedural Language 16
6.1 Procedural Programming (POP) – Basic Idea 16
6.2 Object-Oriented Programming (OOP) – Basic Idea 16
6.3 Detailed Comparison (Procedural vs OOP) 16
6.4 Advantages of OOP Over Procedural 17
6.5 Disadvantages / Limitations of OOP 17
7. Concepts of Message Passing 18
7.1 Definition 18
7.2 Elements of a Message 19
7.3 Message Passing Example 19
7.4 Synchronous vs Asynchronous Message Passing 20
7.5 Message Passing and Encapsulation 20
8. Overview of the Properties of OOPs (Pillars of OOP) 20
8.1 Encapsulation 21
Definition 21
Mechanism 21
Example 21
Benefits 22
8.2 Abstraction 22
Definition 22
Techniques 22
Example – Abstract Class 23
Real-life example 23
Benefits 23
8.3 Inheritance 23
Definition 23
Key Terms 24
Example 24
Types of Inheritance (conceptually) 25
Benefits 25
8.4 Polymorphism 25
Definition 25
Two Major Types 25
Real-life example 27
Benefits 27
9. Additional Examples to Connect All Concepts 27

Example 1: Library System (Conceptual) 27
Example 2: E-Commerce System (Conceptual) 28
10. Practice Questions (MCQ, Short, Long) with Answers 29
Part A: Multiple Choice Questions (MCQs) 29
Part B: Short Answer Questions 30
Part C: Long Answer Questions 31
Answer Key (Brief) 31
Part A: MCQs 31
Part B: Short Answers (Hints) 31
Part C: Long Answers 32

1. Concepts of Object-Oriented Programming Language (OOP)

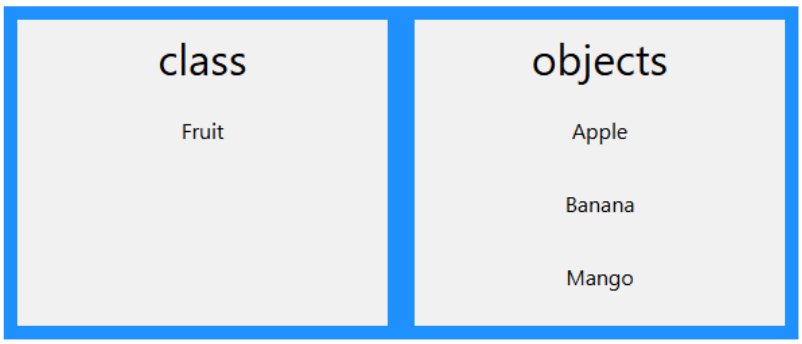
1.1 What is OOP?

- OOP = Object-Oriented Programming
- It is a programming paradigm (style of programming) based on “objects”, not just functions or procedures.
- Objects combine:
  - Data (attributes / fields / variables)
  - Behavior (methods / functions / operations)

In simple words:

- In procedural programming, you think: “Which functions do I need and in what order?”
- In OOP, you think: “What are the real-world entities (objects) involved and how do they interact?”

OOP's concept



1.2 Key Ideas Behind OOP

- World is modeled as objects:
  - Example: In a University system, objects can be: Student, Course, Instructor, Department, Exam, etc.
- Objects have:
  - State (current data)
  - Behavior (operations they can perform)
  - Identity (unique existence in memory)
- Classes are templates/blueprints from which objects are created.
- Program = set of interacting objects that send messages (call methods) to each other.

1.3 Characteristics of an Object-Oriented Language

Typical OOP languages: Java, C++, C#, Python, Smalltalk, Ruby, etc.
Common characteristics they support:

- Classes and Objects
- Encapsulation (data hiding)
- Inheritance (deriving new classes from existing ones)
- Polymorphism (same interface, different implementations)
- Abstraction (showing only essential details)
- Support for relationships between classes (association, aggregation, composition, etc.)

2. Object

2.1 Definition

- An object is:
  - An identifiable entity
  - An instance of a class
  - Having:
    - State (attributes' values)
    - Behavior (methods it can execute)
    - Identity (unique address/handle in memory)

Example:

- Class: Car
- Objects: myCar, yourCar, taxi1, bus2
- Each object will have:
  - Possibly different values of color, speed, fuelLevel, etc.
  - Same methods like drive(), brake(), accelerate()

2.2 State, Behavior, Identity

For a "Student" object:

- State: name = "Rahul", rollNo = 15, cgpa = 8.5
- Behavior: attendClass(), submitAssignment(), giveExam()
- Identity: In languages like Java, internally represented by a reference (memory address), but you only see the variable name.

2.3 Examples

1. Real-world examples:
   - Pen, Pencil, Book, Mobile, BankAccount, Customer, Employee.
   - Each object has its own specific data and behavior.

2. Code-like idea (language-neutral):
   Suppose we have:
   - Class: MobilePhone
   - Object: myPhone

   State:
   - brand = "Samsung"
   - model = "Galaxy S21"
   - batteryLevel = 80

Behavior:

- makeCall(number)
- sendSms(number, message)
- charge()

3. Class

3.1 Definition

- A class is:
  - A user-defined data type
  - A blueprint / template / prototype
  - Defines:
    - What data (attributes) objects of this class will have
    - What behavior (methods) they can perform

A class itself does not hold actual data (like specific student's name). It just defines the structure.

3.2 Analogy

- Class = Architect's plan / design drawing of a house
- Objects = Actual houses built from that plan
- Many houses (objects) can be built from one plan (class).
- Each house may have different color, owner name, etc., but basic structure is same.

3.3 Code-Level Example

```java
class Student {
    // Attributes (data members)
    String name;
    int rollNo;
    double cgpa;
```

```
    // Constructor
    Student(String n, int r, double c) {
        name = n;
        rollNo = r;
        cgpa = c;
    }

    // Methods (behavior)
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
        System.out.println("CGPA: " + cgpa);
    }

    void study() {
        System.out.println(name + " is studying.");
    }
}
```

Using this class to create objects:

```java
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Amit", 101, 8.5);
        Student s2 = new Student("Suman", 102, 9.0);

        s1.display();
        s2.study();
    }
}
```

- Here, `Student` is the class.
- `s1` and `s2` are objects (instances) of that class.

3.4 Types of Classes (Brief Idea)

Languages may support different kinds of classes, e.g.:

- Normal/concrete classes
- Abstract classes (cannot be directly instantiated)
- Final classes (cannot be inherited)
- Inner / nested classes (classes inside another class)
- Meta-classes (we'll discuss later)

4. Relationships Among Objects

Objects in a system do not exist in isolation. They relate and interact with each other. These relationships can be:

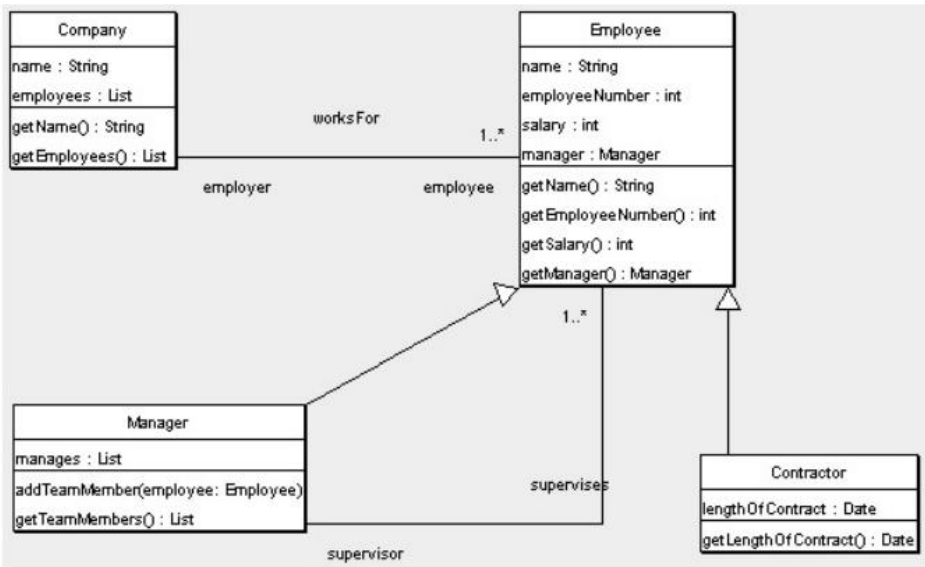
4.1 Types of Object-Level Relationships

Commonly discussed types:

- Association
- Aggregation / Composition (stronger "part-of" relationships)
- Dependency
- Collaboration (for message passing)

High-level overviews of OOP relationships and UML diagrams are in:

OOP CONCEPTS
Classes & Objects
Encapsulation
Getters & Setters
Abstraction, Interfaces & Mixins
Polymorphism
Association, Aggregation & Composition
Static Classes & Static Methods
Generics
Delegates
Methods & Attributes
Access Modifiers
Constructors & Destructors
Inheritance


Company
name : String
employees : List
getName() : String
getEmployees() : List
worksFor
1..*
employer
employee
Employee
name : String
employeeNumber : int
salary : int
manager : Manager
getName() : String
getEmployeeNumber() : int
getSalary() : int
getManager() : Manager
1..*
Manager
manages : List
addTeamMember(employee: Employee)
getTeamMembers() : List
supervises
supervisor
Contractor
lengthOfContract : Date
getLengthOfContract() : Date

4.2 Association Between Objects

- Two objects are associated if one “knows about” the other and can interact.
- Example:
  - A `Teacher` object is associated with a `Student` object.
  - `teacher.teaches(student)`

In code (idea):

```java
class Teacher {
   void teaches(Student s) {
       // teacher uses student object
   }
}

class Student {
   // ...
}
```

- The `Teacher` object uses the `Student` object, but both can exist independently.
- If `Teacher` is deleted, `Student` can still exist (maybe taught by another teacher).

4.3 Aggregation / Whole-Part Between Objects

- Aggregation is a specific kind of association: “has-a” relationship.
- One object contains or holds reference to another object, but the contained object can exist independently.
- Example:
  - A `Department` has `Professor`s.
  - If the `Department` object is destroyed, `Professor` objects can still belong to another department.

4.4 Dependency Between Objects

- Object A depends on Object B if:
  - A needs B to perform some behavior, but
  - A does not “own” B; they are loosely coupled.

- Example:

- A `Order` object depends on a `PaymentGateway` object.
- If `PaymentGateway` changes, `Order` may be affected.

4.5 Interaction Through Message Passing

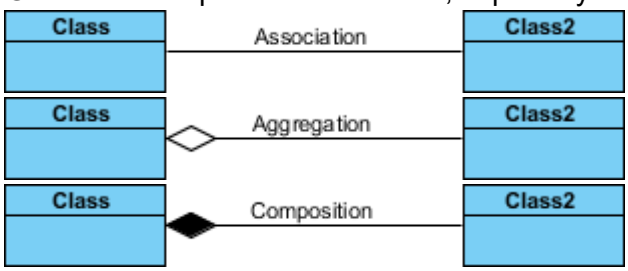
- Objects communicate by calling each other's methods.
- This is called "message passing".
- Example:
  - `atm.withdraw(amount)` sends a message "withdraw" to the `atm` object with argument `amount`.

5. Relationships Among Classes (Class-Level Relationships)

While objects interact at runtime, relationships among classes are design-time decisions captured in class diagrams. Main types:

- Association
- Aggregation
- Composition
- Instantiation
- Generalization/Inheritance
- Dependency
- Meta-class relationships

Good visual explanations of these, especially association vs aggregation vs composition, are here:



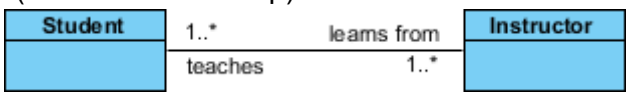
5.1 Association

- Association is a general relationship where two classes are connected.

- Objects of these classes can communicate.
- Represented in UML by a solid line connecting the classes, often with arrows or labels.

Examples:

- Teacher and Student (teaches relationship)



- Customer and Order (places relationship)

Properties:

- Both classes have independent life cycles.
- No “ownership” (no parent–child control).
- Can be one-to-one, one-to-many, many-to-many.

5.2 Aggregation

- Aggregation = “has-a” relationship with weak ownership.
- One class (whole) contains the other (part), but:
  - Part can exist independently.
- UML notation: line with hollow diamond at the “whole” end.

Example:

- `Department` “has-a” list of `Professor`s.
- If `Department` is deleted, `Professor`s can still exist (move to another department).

Pseudo-code:

```java
class Professor {
   String name;
}

class Department {
   String name;
   // Aggregation: department has professors,
   // but professors exist independently
   Professor[] professors;
}
```

5.3 Aggregation Using Instantiation

- Aggregation can be implemented via object creation (instantiation).
- The whole class instantiates objects of the part class and stores references.

Example:

```java
class Engine { ... }

class Car {
   // Car creates and holds reference to an Engine object
   Engine engine = new Engine();
}
```

If we treat this as aggregation (not strict composition), then:

- An `Engine` object could be shared among multiple `Car` objects (unusual, but possible in design).
- `Car` may reference an `Engine` not created by itself but assigned from outside.

5.4 Composition (Strong Aggregation) – for Contrast

Though not explicitly in your syllabus title, understanding composition helps you truly understand aggregation.

- Composition = “has-a” but with strong ownership.
- Part cannot exist without the whole.
- UML notation: line with filled diamond at the whole end.

Example:

- A `Heart` is part of a `Human`. If the `Human` dies, the `Heart` as part of that body does not live independently (conceptually).
- A `File` belongs to a `Folder`. If folder is deleted, file usually is deleted.

5.5 Instantiation Relationship

- Instantiation is the relationship between a class and its object(s).
- “An object is an instance of a class.”
- In some models, you can show a dependency or dashed arrow from object (or a creator class) to the class being instantiated.

Examples:

- `new Student()` creates an instance of class `Student`.
- In a factory pattern, a `StudentFactory` class creates `Student` objects.

5.6 Meta-Class

Definition:

- A meta-class is a class whose instances are themselves classes.
- A normal class defines properties and behavior of objects.
- A meta-class defines properties and behavior of classes (e.g., how classes behave, how they are created).

Simple mental model:

- Object ← instance of ← Class ← instance of ← Meta-class
- Meta-class is “class of a class”.

Languages where meta-classes are explicitly available:

- Python, Smalltalk, Ruby, etc.

Example idea (high-level):

- In Python, `type` is a meta-class.
- When you write:

```python
class MyClass:
    pass
```

- Python actually uses the meta-class `type` to construct `MyClass`.

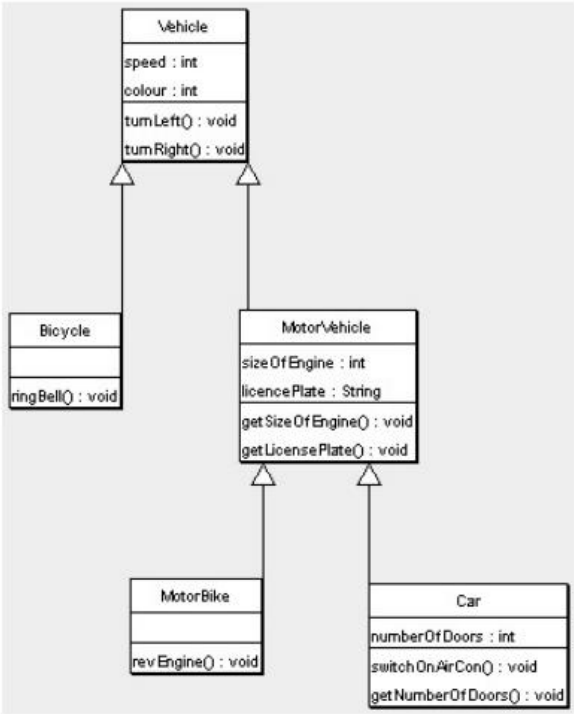
Meta-classes are used for:

- Advanced behaviors like:
  - Automatically registering classes
  - Enforcing coding standards
  - Changing how classes are created
  - Implementing singleton patterns, etc.

5.7 Other Class Relationships (Brief)

- Generalization (Inheritance):
  - “is-a” relationship.
  - Car is a Vehicle.
- Realization (Interface implementation):
  - A class implements an interface.
- Dependency:

- One class uses another (e.g., as a parameter or local variable).



6. Differences Between OOP and Procedural Language

6.1 Procedural Programming (POP) – Basic Idea

- Focus: Functions / procedures / subroutines.
- Data (variables) is often separate from functions.
- Program = sequence of instructions organized into functions.
- Top-down design: break problem into smaller steps/functions.

Examples of languages:

- C, Pascal, FORTRAN, BASIC, etc.

6.2 Object-Oriented Programming (OOP) – Basic Idea

- Focus: Objects that combine data and behavior.
- Program = set of objects that interact via methods.
- Bottom-up design: first identify objects/classes, then their interactions.

Examples:

- Java, C++, C#, Python, Smalltalk, Ruby

6.3 Detailed Comparison (Procedural vs OOP)

Detailed comparison table:

ASPECT	PROCEDURAL PROGRAMMING (POP)	OBJECT-ORIENTED PROGRAMMING (OOP)
Focus	Functions / procedures and steps	Objects and their interactions
Decomposition	Top-down (problem → functions → subfunctions)	Bottom-up (identify objects/classes → behavior)
Data & Functions	Treated separately; global data is common	Data and functions encapsulated together in objects
Security / Data Hiding	Weak; data can be accessed globally	Strong via encapsulation and access modifiers
Reusability	Difficult; often copy-paste or rewrite code	High via inheritance and polymorphism
Abstraction	Procedural abstraction (functions)	Data abstraction (classes + objects)
Maintenance & Scaling	Suitable for small-medium systems; harder to maintain	Better for large, complex systems; modular design

Communication	Direct function calls with parameters	Objects communicate via message passing (method calls)
Example Languages	C, Pascal, FORTRAN, BASIC	Java, C++, C#, Python, Smalltalk

6.4 Advantages of OOP Over Procedural

1. Modularity:
    - System is divided into smaller, self-contained objects.
    - Easier to understand, debug, and maintain.
2. Reusability:
    - Inheritance allows new classes to reuse code of existing classes.
    - Libraries and frameworks can be extended easily.
3. Data Security:
    - Encapsulation restricts unauthorized access.
    - Data can be made private and accessed only through public methods.
4. Easier Design for Complex Systems:
    - Real-world mapping makes design more natural.
    - Easier to represent domains like banking, e-commerce, university, etc.
5. Flexibility and Extensibility:
    - Polymorphism allows new behaviors without modifying existing code much.
    - New classes can be added with minimal changes.
6. Better Teamwork:
    - Classes and interfaces can be assigned to different developers.
    - Clear boundaries between components.

6.5 Disadvantages / Limitations of OOP

1. Larger Program Size:
    - More classes and objects lead to more lines of code compared to small procedural programs.
    - Overhead of defining classes, methods, etc.
2. Slower Execution (slightly):
    - More layers of indirection (object references, dynamic dispatch).
    - May matter in extremely performance-critical systems.
3. Steep Learning Curve:
    - Concepts like inheritance, polymorphism, abstract classes, interfaces can be confusing for beginners.

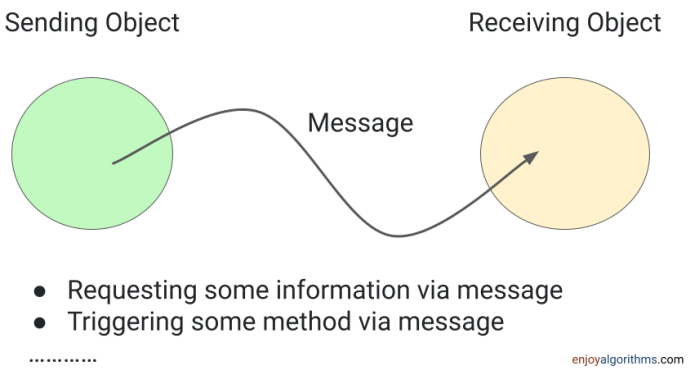
- Requires good design skills.

4. Overhead for Small Programs:
   - For very small scripts or simple utilities, full OOP may feel like “overkill”.
5. Complexity in Design:
   - Poorly designed class hierarchies can make the system hard to understand.
   - Misuse of inheritance can cause fragile base class problem.

7. Concepts of Message Passing

7.1 Definition

- Message passing is a way for objects to communicate.
- In OOP, “sending a message” = calling a method on an object.
- The calling object (sender) asks the receiving object (receiver) to perform some action.



7.2 Elements of a Message

A message typically includes:

1. Name of the object (receiver)
2. Name of the method (operation)
3. Information / arguments (parameters)

Example:

```java
account.deposit(5000);
```

- `account` – object receiving the message
- `deposit` – method name (message type)
- `5000` – argument (data required for the operation)

7.3 Message Passing Example

```java
class Account {
   private double balance;

   public Account(double initialBalance) {
       balance = initialBalance;
   }

   // Method that represents a message
   public void deposit(double amount) {
       balance = balance + amount;
       System.out.println("Deposited: " + amount);
   }

   public void displayBalance() {
       System.out.println("Balance: " + balance);
   }
}

class Customer {
   public void transact() {
       Account myAccount = new Account(1000);

       // Sending messages to myAccount object
       myAccount.deposit(500);
       myAccount.displayBalance();
   }
}
```

- Here, `Customer` object sends messages `deposit(500)` and `displayBalance()` to `Account` object.

7.4 Synchronous vs Asynchronous Message Passing

In many OOP languages:

- Synchronous:
  - Sender waits until receiver finishes the method and returns.
  - Normal method calls in Java/C++ are synchronous.
- Asynchronous:
  - Sender sends message and continues without waiting.
  - Used in concurrent/distributed systems (not always in basic OOP course).

7.5 Message Passing and Encapsulation

- Message passing enforces encapsulation:
  - One object cannot directly access another object's private data.
  - It must request the other object to perform an action via its public interface.
- This reduces unwanted side effects and dependencies.

---

8. Overview of the Properties of OOPs (Pillars of OOP)

The major properties/pillars of OOP are usually listed as:

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

Some sources also include:

- Association, Aggregation, Composition (class relationships)
- Dynamic binding
- Message passing

We'll discuss each pillar in detail.

8.1 Encapsulation

Definition

- Encapsulation = bundling data (variables) and methods (functions) into a single unit (class) and restricting direct access to some components.
- Also known as “data hiding”.

Mechanism

- Use access modifiers:
  - `private`, `protected`, `public` etc. (language-specific)
- Keep data variables private or protected.
- Provide public getter and setter methods to read/modify data.

Example

```java
class Employee {
   private String name;
   private double salary;

   public Employee(String n, double s) {
       name = n;
       salary = s;
   }

   public String getName() {
       return name;
   }

   public void setSalary(double s) {
       if (s > 0) {
           salary = s;
       }
   }

   public double getSalary() {
       return salary;
   }
```

```
}
```

- Here, `name` and `salary` are private.
- Outside code cannot modify `salary` directly, only through `setSalary()` which can validate the value.

Benefits

- Protects internal state from invalid changes.
- Hides implementation details from outside world.
- Easier to change internal implementation without breaking other code.

Real-life analogy:

- A capsule (medicine):
  - You only know what to do with it (swallow), not how chemicals work inside.
  - API of a class is like the capsule shell.

8.2 Abstraction

Definition

- Abstraction = showing only essential features and hiding unnecessary details.
- Focus on “what” an object does rather than “how”.

Techniques

- Abstract classes
- Interfaces (in Java, C#, etc.)

Example – Abstract Class

```java
abstract class Shape {
   abstract void draw();  // only declaration, no implementation
}

class Circle extends Shape {
   void draw() {
       System.out.println("Drawing a Circle");
   }
}

class Rectangle extends Shape {
```

```
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}
```

- `Shape` defines that every shape can be drawn, but hides how.
- Each subclass provides its own implementation.

Real-life example

- Car steering wheel:
  - Driver knows turning the wheel changes direction.
  - Driver does not need to know internal steering mechanism (rack-and-pinion, power steering, etc.).

Benefits

- Simplifies complexity.
- Allows design to evolve without affecting users of the abstraction.

8.3 Inheritance

Definition

- Inheritance = a mechanism where a new class (child / subclass) derives attributes and behaviors from an existing class (parent / superclass).
- Establishes “is-a” relationship.

Key Terms

- Superclass / Base class / Parent class
- Subclass / Derived class / Child class

Example

```java
class Vehicle {
    String brand;
    int speed;

    void start() {
        System.out.println("Vehicle started");
    }
}
```

```java
class Car extends Vehicle {
    int numberOfDoors;

    void honk() {
        System.out.println("Car horn: Beep Beep!");
    }
}

public class Test {
    public static void main(String[] args) {
        Car c = new Car();
        c.brand = "Toyota";
        c.numberOfDoors = 4;
        c.start();    // inherited from Vehicle
        c.honk();     // own method
    }
}
```

- `Car` inherits `brand` and `start()` from `Vehicle`.
- Code reuse: common behavior placed in `Vehicle`, specialized behavior in `Car`.

Types of Inheritance (conceptually)

- Single inheritance
- Multiple inheritance (supported in C++, but not directly in Java using classes)
- Multilevel inheritance
- Hierarchical inheritance
- Hybrid inheritance

Benefits

- Reusability of code.
- Logical structure and hierarchical classification.
- Easy to extend functionality.

8.4 Polymorphism

Definition

- Polymorphism = “many forms”.

- Same interface (method name) can behave differently based on object or parameters.

Two Major Types

1. Compile-time Polymorphism (Static Binding)
    - Achieved by:
        - Method overloading - Same method name, different parameter list.

Example:

```java
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

- `add(2, 3)` calls first method.
- `add(2.5, 3.1)` calls second method.

2. Runtime Polymorphism (Dynamic Binding)
    - Achieved by:
        - Method overriding - Same method name and parameters in parent and child; child provides specific implementation.

Example:

```java
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void makeSound() {
        System.out.println("Dog barks");
    }
}
```

```java
class Cat extends Animal {
    void makeSound() {
        System.out.println("Cat meows");
    }
}

public class Test {
    public static void main(String[] args) {
        Animal a;
        a = new Dog();
        a.makeSound(); // Output: Dog barks

        a = new Cat();
        a.makeSound(); // Output: Cat meows
    }
}
```

- Same reference `a` of type `Animal` can refer to `Dog` or `Cat`, and the appropriate method is called at runtime.

Real-life example

- "Play" button on a media player:
  - If audio file selected → plays sound.
  - If video file selected → plays video.
- Same action ("Play"), different behavior.

Benefits

- Flexibility and extensibility.
- Cleaner code using generic interfaces.

---

9. Additional Examples to Connect All Concepts

Example 1: Library System (Conceptual)

Classes:

- `Book`

- `Member`
- `Library`

Relationships:

- Association:
  - `Member` borrows `Book`.
- Aggregation:
  - `Library` has many `Book` objects (books can exist even if library is destroyed, at least conceptually).
- Encapsulation:
  - `Book` keeps fields like `isbn`, `title`, `author` private.
  - Methods like `borrow()` and `returnBook()` modify status.
- Inheritance:
  - `Member` class, with subclasses `StudentMember` and `FacultyMember` (different borrowing limits).
- Polymorphism:
  - `calculateFine()` method overridden in subclasses.
- Message passing:
  - `library.issueBook(member, book)` sends messages to `member` and `book` objects to update their states.

Example 2: E-Commerce System (Conceptual)

Classes:

- `User`
- `Product`
- `Cart`
- `Order`

Relationships:

- Association:
  - `User` places `Order`.
- Aggregation:
  - `Order` has multiple `Product`s.
- Composition (if considered):
  - `Cart` exists only while a `User` is shopping; once session ends, cart may be destroyed.
- Encapsulation:
  - `Order` maintains payment details privately.
- Inheritance:

- `User` as parent; `Customer` and `Admin` as subclasses.
- Polymorphism:
  - `pay()` method with different modes: `payByCard()`, `payByUPI()`, etc. (overloading/overriding).
- Message passing:
  - `cart.add(product)` sends message to `cart` to add a `Product`.

---

10. Practice Questions (MCQ, Short, Long) with Answers

Part A: Multiple Choice Questions (MCQs)

1. Which of the following is NOT a fundamental pillar of OOP?
   a) Encapsulation
   b) Polymorphism
   c) Compilation
   d) Inheritance

2. A class is:
   a) An instance of an object
   b) A blueprint for creating objects
   c) A collection of only functions
   d) A runtime entity

3. The relationship where a class has another class, but the part can exist independently, is called:
   a) Composition
   b) Aggregation
   c) Inheritance
   d) Instantiation

4. Which OOP concept is used to hide implementation details and show only functionality?
   a) Encapsulation
   b) Abstraction
   c) Polymorphism
   d) Inheritance

5. Message passing in OOP is implemented via:
   a) Direct variable access
   b) Global functions
   c) Method calls
   d) Compiler directives

6. Which of the following is an example of a procedural programming language?
   a) Java
   b) C++
   c) C
   d) Python

7. A meta-class is:
   a) A class with no methods
   b) A class whose instances are classes
   c) A superclass for all interfaces
   d) A class without any objects

8. In OOP, the term “object” refers to:
   a) Only data
   b) Only methods
   c) An instance of a class with state and behavior
   d) A template for creating classes

9. Which language feature is mainly used to achieve runtime polymorphism?
   a) Method overloading
   b) Method overriding
   c) Constructor chaining
   d) Static methods

10. In `obj.methodName(10, 20)`, `10` and `20` are called:
    a) Objects
    b) Classes
    c) Messages
    d) Arguments

Part B: Short Answer Questions

1. Define an object and a class. How are they related?
2. What is encapsulation? How is it achieved in OOP?

3. Differentiate between aggregation and composition.
4. What is a meta-class? In what kind of languages is it typically used?
5. What is message passing in OOP? Give a small example.
6. List any four advantages of OOP over procedural programming.
7. State any two disadvantages of OOP.
8. Explain abstraction with a real-world example.
9. What is method overloading? Is it compile-time or runtime polymorphism?
10. What is method overriding? Give a short example.

Part C: Long Answer Questions

1. Explain the four major properties (pillars) of OOP in detail with examples and one diagram idea for each.
2. Compare and contrast procedural programming and object-oriented programming. Focus on focus, approach, data handling, reusability, and suitability for different types of applications.
3. Discuss the following relationships among classes with suitable examples:
   a) Association
   b) Aggregation
   c) Composition
   d) Inheritance
4. What is message passing in OOP? Explain how message passing helps in achieving encapsulation and modularity. Support your answer with a program-like example.
5. Define meta-class. Explain its role in OOP languages with examples of languages that support meta-classes. How does a meta-class differ from a normal class?

Answer Key (Brief)

Part A: MCQs

1. c) Compilation
2. b) A blueprint for creating objects
3. b) Aggregation
4. b) Abstraction
5. c) Method calls
6. c) C
7. b) A class whose instances are classes
8. c) An instance of a class with state and behavior
9. b) Method overriding

10. d) Arguments

Part B: Short Answers (Hints)

1. Class is a blueprint; object is an instance of a class.
2. Encapsulation = bundling data and methods and restricting access via access modifiers.
3. Aggregation: weak “has-a”, part can exist independently; Composition: strong “has-a”, part depends on whole.
4. Meta-class: a class whose instances are classes (e.g., `type` in Python).
5. Message passing: calling methods on objects (e.g., `account.deposit(1000)`).
6. Advantages: reusability, modularity, data security, easier maintenance.
7. Disadvantages: larger program size, slower, steeper learning curve.
8. Abstraction: showing only necessary features, like using a TV via remote instead of knowing internal circuits.
9. Overloading = same name, different parameters (compile-time polymorphism).
10. Overriding = same name and signature in parent and child (runtime polymorphism).

Part C: Long Answers

For long answers, refer to Sections 5 to 9.

End of Module 1